Go Rewrite Implementation Plan

Session runtime first. CLI as an adapter. Future sandboxd as an orchestration layer.

Architectural rule
pi-sesh owns the safe execution of one sandboxed session operation. It must not depend on who requested that operation or on multi-session orchestration. A future sandboxd should reuse the same session runtime rather than reimplement it. pi-sesh is Git-agnostic: it accepts a supplied local directory; Git provisioning belongs to future sandboxd workspace providers, and push/PR/GitHub workflows belong to the orchestrator/integration layer.

Scope: rewrite the existing TypeScript pi-sesh implementation in Go while preserving its security and lifecycle invariants.

Prepared from the current pi-sesh design and the agreed CLI/runtime/control-plane split.

1. Target architecture

The rewrite should be organized around a reusable session runtime. The CLI is one frontend. A daemon can be added later without moving lifecycle logic.

```
pi-sesh CLI
    |
    v
Session Manager
    |-- Store
    |-- Workspace
    |-- Resources
    v
Runtime interface
    |
    v
Podman
    |
    v
Process runner
```

Future extension

```
              pi-sesh CLI
                   |
                   v
           Session Manager <----- sandboxd
                   |                 |
                   v                 +-- scheduling
                Runtime              +-- leases
                   |                 +-- queues/events
                   v                 +-- supervision
                 Podman
```

sandboxd should add coordination over time: scheduling, concurrency limits, leases, heartbeats, queues, event streams, recovery, and multi-client coordination. It should reuse pi-sesh filesystem mount/copy preparation, Podman argument construction, session cleanup, and resource safety. Git workspace acquisition belongs to a future sandboxd workspace provider.

2. Proposed repository structure

```
pi-sesh/
|-- cmd/
|   `-- pi-sesh/
|       `-- main.go
|-- internal/
|   |-- cli/
|   |-- session/
|   |-- runtime/
|   |-- podman/
|   |-- workspace/
|   |   |-- manager.go
|   |   |-- mount.go
|   |   `-- copy.go
|   |-- resource/
|   |-- config/
|   `-- process/
|-- integration/
|-- docs/
|-- AGENTS.md
`-- go.mod
```

The main dependency direction is CLI -> Session Manager -> domain modules -> Runtime -> Podman -> Process. Lower layers must not import CLI concerns.

3. Module ownership

Module	Owns	Does not own
process	Safe host subprocess execution	Podman or session semantics
runtime	Container-runtime contract and neutral types	Podman implementation details
podman	Rootless Podman mechanics and argv construction	Session lifecycle
workspace	Safe mount/copy preparation and cleanup	Container lifecycle or Git provisioning
resource	Snapshots, owned directories, secrets, staging and cleanup	Global session orchestration
session.Store	Persistent session metadata and lookup	Runtime truth
session.Manager	Lifecycle ordering, invariants and rollback	CLI, HTTP, scheduling
config	Strict parsing and trust-boundary validation	Execution
cli	Flags, terminal IO, formatting, exit codes	Business logic
future api	Transport into Session Manager	Lifecycle behavior
future sandboxd	Multi-session coordination, workspace acquisition and supervision	Sandbox implementation or PR workflows

4. Implementation sequence

Build from the lowest-level stable boundaries upward. Do not begin by recreating the complete CLI. Each step should leave a testable component behind.

Step 1 - Freeze the behavioral contract

Focus: Document the current TypeScript behavior before changing architecture.

- Create docs/behavior.md and list security, lifecycle, workspace, resource, and cleanup invariants.
- Classify existing features as Go v1, deferred, or removed. Record clone removal and copy introduction as an intentional contract change; retain all unrelated security and lifecycle guarantees.
- Keep the TypeScript version as the executable reference during migration.

Step 2 - Implement internal/process

Focus: Create one safe subprocess boundary used by Podman.

- Use exec.CommandContext with literal argv only.
- Own stdin/stdout/stderr wiring, working directory, sanitized environment, exit-code extraction, cancellation, timeouts, and bounded output capture.
- Never accept reconstructed shell command strings.

Step 3 - Define domain types

Focus: Establish stable session vocabulary before orchestration.

- Define SessionID, WorkspaceMode, NetworkMode, and lifecycle states.
- Make UUID-like session IDs the canonical internal identity; human names are aliases.
- Separate persisted metadata from live runtime state: Podman remains authoritative about whether a container is actually running.

Step 4 - Implement session.Store

Focus: Move persistence out of lifecycle orchestration.

- Create/update/get/resolve/list/delete session records.
- Preserve the current state layout where practical to ease migration.
- Use atomic writes, safe permissions, versioned records, corruption handling, and name collision checks.
- Leave a clean seam for per-session filesystem locking.

Step 5 - Define the Runtime interface

Focus: Make Session Manager depend on capabilities, not Podman flags.

- Expose image resolution, create/start/inspect/exec/logs/stop/remove.
- Keep Podman JSON structs, flags, network internals, and volume syntax out of the interface.
- Keep the interface deliberately small; this boundary is for architecture and testing, not speculative Docker support.

Step 6 - Port Podman

Focus: Implement Runtime using the Podman CLI.

- Keep rootless operation and CLI invocation; do not switch to exposing a Podman socket.
- Port preflight checks, local-image validation, sanitized environment, network validation, inspect parsing, and explicit argv generation.
- Put argv construction in pure functions so it can be unit tested heavily.

Step 7 - Port workspace handling

Focus: Isolate filesystem mount and copy preparation from session lifecycle.

- workspace.Manager.Prepare returns a prepared host path, mount path, mode, and ownership/cleanup metadata; it has no repository or source-revision fields.
- Keep mount validation small and explicit. Mount uses a caller-supplied directory without taking ownership; copy creates a pi-sesh-owned session snapshot. Cleanup must never delete the supplied source or a provider-owned checkout.
- Copy snapshots the supplied directory using filesystem rules. Define symlink handling, limits, permissions, cancellation, and rollback; validate path and ownership before cleanup.
- Use workspace/manager.go, mount.go, and copy.go. Remove clone.go and git.go from the pi-sesh module plan; no Git subprocesses belong in this module.

Step 8 - Port resource handling

Focus: Move existing handoff behavior into an explicit resource subsystem.

- Rename the internal concept to resource while keeping user-facing terminology compatible if desired.
- Model snapshots, owned directories, retained resources, and secret resources.
- Preserve symlink traversal protection, size/file-count limits, ownership checks, inode/device verification, destination collision checks, and secret cleanup.

Step 9 - Implement session.Manager

Focus: Create the core pi-sesh runtime orchestration layer.

- Start should read like a transaction: validate -> preflight -> resolve image -> reserve identity -> prepare workspace -> stage resources -> create container -> persist -> start.
- Every step must have explicit rollback behavior.
- Remove should fail closed: verify runtime and ownership before deleting resources, owned copy state, or metadata.
- Manager must know nothing about Cobra, JSON rendering, HTTP, queues, or global orchestration.

Step 10 - Add typed errors

Focus: Make errors useful to both CLI and a future API.

- Define stable categories such as ErrNotFound, ErrAlreadyExists, ErrInvalidState, ErrUnsafeCleanup, and ownership errors.
- Wrap errors with context while preserving errors.Is compatibility.
- A future API should be able to map these directly to transport status without parsing strings.

Step 11 - Build the minimal CLI

Focus: Add the thin human-facing adapter only after the core works.

- Implement start, exec, inspect, list, logs, stop, remove, and open.
- CLI handlers only parse flags, build request structs, call Session Manager, render results, and map errors to exit codes.
- If a command handler becomes large, move the behavior down into the core rather than creating CLI services.

Step 12 - Add machine-readable output

Focus: Make the CLI useful for automation before sandboxd exists.

- Add --json for start, inspect, list, and other appropriate commands.
- Keep output formatting outside domain types.
- Treat the CLI process interface as the stable integration surface for Go v1 rather than recreating a TypeScript SDK immediately.

Step 13 - Port project config

Focus: Preserve existing pi-sesh.json behavior, with the explicit Git boundary change.

- Use strict JSON decoding and reject unknown fields.
- Treat project configuration as untrusted input.
- Preserve unrelated config behavior. Replace Git clone mode with filesystem copy; reject obsolete Git options explicitly rather than silently treating clone as copy.

Step 14 - Port trusted launch presets

Focus: Keep trusted host configuration separate from project config.

- User-owned presets may contain host preparation, runtime mounts, resource configuration, and defaults.
- Use separate types for ProjectConfig and LaunchPreset to preserve the trust boundary.
- Do not allow project configuration to gain arbitrary host-command execution.

Step 15 - Add interactive open behavior

Focus: Keep terminal mechanics at the edge.

- Raw terminal mode, TTY detection, resizing, Ctrl-C presentation, and stdin/stdout attachment belong to the CLI.
- The session/runtime layers should expose neutral operations or streams rather than terminal UX.

Step 16 - Define cancellation semantics

Focus: Make context.Context meaningful end to end.

- Propagate command context through Manager, workspace/resource operations, Runtime, and process.Run.
- For exec timeout/cancellation, preserve conservative behavior where uncertain process state can trigger sandbox shutdown.
- Document cancellation and cleanup guarantees rather than relying on incidental exec.CommandContext behavior.

Step 17 - Add concurrency safety

Focus: Protect session mutations even before sandboxd exists.

- Use a per-session lock around mutating operations such as exec/stop/remove where races matter.
- A filesystem lock is sufficient for multiple CLI processes and remains useful if sandboxd is later introduced.
- Define idempotent behavior where practical, for example stopping an already stopped session.

Step 18 - Run compatibility tests

Focus: Use the TypeScript implementation as the migration oracle.

- Compare mount starts against TypeScript; test copy starts against the new filesystem contract. Cover network none/pasta, environment forwarding, resource types, exec behavior, stop/remove, missing containers, and unsafe cleanup.
- Compare Podman argv, mounts, environment, persisted metadata, filesystem permissions, cleanup results, and exit semantics - not only CLI text.

Step 19 - Add real Podman integration tests

Focus: Keep real runtime tests focused and opt-in.

- Gate with an environment variable such as PI_SESH_PODMAN_TESTS=1.
- Test create/start/exec/inspect/stop/remove plus representative workspace, network, and resource behavior.
- Keep the majority of coverage in unit and fake-runtime tests.

Step 20 - Cut over to Go

Focus: Make the Go implementation canonical once core parity is reached.

- Deprecate the TypeScript implementation rather than evolving both indefinitely.
- Where practical, keep enough session-state compatibility that Go can inspect or safely clean sessions created by the old implementation.

Step 21 - Port deferred convenience features

Focus: Add non-core functionality after the architecture is stable.

- Build/build-default, clean, stop-all, attach, setup-prompt, preset conveniences, and richer human output should not block runtime replacement.

Step 22 - Verify the sandboxd seam

Focus: Test the architecture by imagining a second adapter, not by building the daemon.

- A future cmd/sandboxd should be able to construct Session Manager and expose it through HTTP or a Unix socket without importing internal/cli.
- sandboxd may add scheduling, leases, queues, heartbeats, event subscriptions, global concurrency limits, garbage collection, and supervision.
- It must reuse the same session lifecycle implementation.

5. Core API shape

The reusable surface should be request/response based and transport-agnostic. One reasonable shape is:

```
type Manager struct {
    Runtime    runtime.Runtime
    Store      Store
    Workspaces *workspace.Manager
    Resources *resource.Manager
}

func (m *Manager) Start(ctx context.Context, req StartRequest) (Session, error)
func (m *Manager) Inspect(ctx context.Context, id ID) (SessionView, error)
func (m *Manager) Exec(ctx context.Context, id ID, req ExecRequest) (ExecResult, error)
func (m *Manager) Stop(ctx context.Context, id ID) error
func (m *Manager) Remove(ctx context.Context, id ID) error
func (m *Manager) List(ctx context.Context) ([]SessionView, error)
```

Avoid APIs shaped around CLI args or writers. The same manager should be callable from Cobra handlers, tests, or a future local API.

6. Start and remove transaction boundaries

Start

```
validate request
    -> runtime preflight
    -> resolve local image
    -> reserve session identity
    -> prepare workspace
    -> stage resources
    -> create container
    -> persist session record
    -> start container
```

Every transition that allocates host or runtime state must define rollback. The main benefit of the rewrite is not fewer lifecycle steps; it is making these steps visible and local.

Remove

```
resolve session
    -> inspect runtime
    -> verify safe removal / ownership
    -> stop or remove container as appropriate
    -> clean secret and retained resources safely
    -> remove pi-sesh-owned copy workspace
    -> remove session metadata / name alias
```

Cleanup should continue to fail closed. If ownership or identity cannot be proven, leave the resource in place and return an explicit error.

7. Security invariants to preserve

- Rootless Podman only.
- Podman CLI boundary rather than exposing a host Podman socket to sessions.
- Literal argv; never reconstruct untrusted input into shell commands.
- Project configuration is untrusted.
- Trusted user presets are the only place for host preparation behavior.
- No implicit image pulls during normal session start.
- Network disabled by default; explicit opt-in modes only.

- No host home, container-engine socket, or broad credential mounting.
- Explicit environment forwarding.
- Ownership, path, inode/device, size, and file-count checks around staged resources.
- Secret cleanup semantics remain conservative.
- Unsafe cleanup fails closed.

8. Testing strategy

Layer 1 - Pure unit tests

- Config parsing and trust validation.
- Session store behavior.
- Workspace path validation, mount/copy ownership, copy rollback, and source preservation.
- Resource manifest validation.
- Podman argv generation and inspect conversion.
- Typed error and state-transition behavior.

Layer 2 - Fake runtime lifecycle tests

Exercise Session Manager with a fake Runtime and temporary filesystem. Cover create/start failures, rollback, missing containers, ownership mismatch, cleanup failure, name collisions, cancellation, and concurrent mutations.

Layer 3 - Real Podman integration tests

Keep a small opt-in suite for the actual process/runtime boundary. It should prove that the generated Podman operations behave as intended on a real rootless engine.

9. Suggested milestone breakdown

Milestone	Scope	Exit condition
M0 - Contract	Behavior doc + AGENTS invariants	Clear v1/deferred scope and parity scenarios exist.
M1 - Foundations	process, domain types, store, runtime interface	Unit-tested lower layers with no CLI dependency.
M2 - Podman core	Podman preflight/image/create/inspect/start/stop/remove	Can create and remove a minimal container through Runtime.
M3 - Session lifecycle	Manager start/inspect/stop/remove + mount workspace	A useful Go runtime exists without convenience features.
M4 - Full sandbox core	copy workspace, resources, exec, logs, cancellation	Core sandbox behavior is available safely, with the explicit workspace contract change.
M5 - CLI parity	minimal commands + --json + project config	agent-orchestrator can use the Go binary.
M6 - Compatibility	TS parity suite + copy contract tests + real Podman integration suite	Go implementation is safe to make canonical.

Milestone	Scope	Exit condition
M7 - Convenience	presets/build/attach/cleanup helpers as needed	Remaining useful features migrate incrementally.

10. Definition of the first useful Go release

Do not define success as "all TypeScript features rewritten." The first useful release is reached when the following workflow is reliable:

```
pi-sesh start .
pi-sesh exec my-session -- git status
pi-sesh inspect my-session
pi-sesh stop my-session
pi-sesh remove my-session
```

- Mount and copy workspaces work.
- Network none/pasta behavior is preserved.
- Only local validated images are used for normal session start.
- Project configuration is supported safely.
- Snapshot/secret/owned-directory resource handling works.
- Cleanup and ownership invariants are preserved.
- The CLI is thin and the Session Manager can be reused independently.

11. Architectural acceptance criteria

- No package below internal/cli imports CLI-specific types or Cobra.
- No Session Manager method accepts raw CLI args or formats terminal output.
- Podman-specific structs do not escape internal/podman.
- Workspace and resource modules can be tested without a container runtime.
- Session lifecycle can be tested with a fake Runtime.
- The persistent Store is independent of Podman.
- A second adapter could call Session Manager without modifying lifecycle code.
- Concurrency protection exists at the session mutation boundary.
- Existing security invariants have explicit tests or documented checks.
- pi-sesh has no Git-specific workspace modes, repository/ref requests, or Git subprocess dependency. Git tools may still run inside the container through ordinary exec.

12. Deferred sandboxd responsibilities

Do not implement these in pi-sesh merely to future-proof it. They belong in a later control plane if actual requirements appear:

- Scheduling and job queues.
- Global or per-project concurrency limits.

- Leases and heartbeats.
- Multi-client ownership and coordination.
- Event subscriptions and live status streams.
- Abandoned-session detection and garbage collection.
- Long-running supervision and recovery policy.
- Cross-session resource accounting and admission control.

Design test

If sandboxd can later be implemented as a thin adapter that depends on Session Manager, the rewrite has the right boundary. If sandboxd would need to import CLI code or reproduce Podman or pi-sesh mount/copy/resource behavior, the boundaries are wrong. Workspace acquisition policy remains in sandboxd.

12a. Future sandboxd workspace providers

The future control plane acquires a workspace before calling pi-sesh. This is deferred work, not a new pi-sesh dependency.

Orchestrator/integration -> sandboxd workspace provider -> local directory -> pi-sesh session runtime -> container

A WorkspaceProvider can support a local directory or Git-backed checkout. The Git provider owns clone/fetch, revision selection, checkout policy, repository metadata, and any submodule or sparse-checkout policy. Preserve the former clone hardening here: disable hooks and implicit credentials, sanitize Git configuration, validate paths and ownership, and define cleanup and cancellation behavior. Any required authentication must be explicitly scoped by the integration.

The provider returns a validated local path and provider-owned lifecycle metadata to sandboxd. sandboxd passes only the local directory and mount/copy policy to pi-sesh; repository URLs, refs, and source revisions remain outside the runtime API. Provider metadata may retain a resolved revision for provenance.

For mount, sandboxd retains ownership of its checkout and releases it only after the session is safely removed. For copy, pi-sesh owns its snapshot and sandboxd owns the provider source; each layer rolls back and cleans only its own allocations. Define release/retention policy explicitly, including provision failures and session-start failures.

The orchestrator/integration owns task intent, branch/publish policy, push authorization, GitHub APIs, and PR creation. The agent may use Git inside the sandbox through normal command execution, subject to explicit network and credential policy. pi-sesh does not interpret these commands or model their repository semantics.

Future acceptance checks: provider failures do not create sessions; failed starts release provider allocations safely; session removal preserves externally owned sources; Git provenance never becomes a pi-sesh runtime requirement.

13. Recommended implementation order - concise checklist

1. Freeze behavior and security contract.

2. Implement safe process runner.

3. Define domain types and typed errors.

4. Implement file-backed session store.

5. Define Runtime interface.

6. Implement Podman Runtime.

7. Implement workspace mount and copy.

8. Implement resource staging and cleanup.

9. Implement Session Manager start/inspect/stop/remove.

10. Add exec/logs/open and cancellation semantics.

11. Add per-session locking.

12. Add thin CLI and --json output.

13. Port project config and trusted presets.

14. Run TS parity scenarios and real Podman tests.

15. Cut over to Go.

16. Port convenience features only as needed.

17. Keep sandboxd and Git workspace provisioning out of Go v1 until orchestration requirements justify them.

End state

A small Go session runtime with explicit lifecycle semantics, strong cleanup/security boundaries, a thin CLI, and a clean reusable seam for a future orchestration daemon.